



BITS Pilani
Pilani Campus

Natural Language Processing Applications

Dr. Chetana Gavankar, Ph.D,
IIT Bombay-Monash University Australia
Chetana.gavankar@pilani.bits-pilani.ac.in



BITS Pilani
Pilani Campus

Session 3:

These slides are prepared by the instructor, with grateful acknowledgement of all those who made their course materials freely available online.

Agenda – What We'll Cover



01

RAG Foundations

Why RAG, core architecture, NLP context, key components

02

Naïve RAG Deep Dive

Classic pipeline, chunking, retrieval, embeddings & limitations

03

Graph RAG

Knowledge graphs, entity extraction, community summaries

04

Agentic RAG

Autonomous agents, tool use, planning & multi-step reasoning

05

Real-World Case Studies

Healthcare, Legal, Finance, Customer Service deployments

06

Latest Research (2024-25)

Frontier papers, benchmarks, open problems & future directions

RAG Foundations – What & Why



What is RAG?

RAG augments large language models by dynamically retrieving relevant external knowledge at inference time – grounding answers in verifiable, up-to-date information without costly retraining.

Problems RAG Solves in NLP

- ✗ **Hallucination** – LLMs confabulate facts not in training
- ✗ **Knowledge cutoff** – Static weights miss recent events
- ✗ **No citations** – Cannot trace generation to sources
- ✗ **Domain blind spots** – General models lack specialist depth

Core RAG Pipeline



✓ ↓ 40-60% hallucinations

✓ ↑ Source traceability

✓ No fine-tuning needed

✓ Real-time knowledge

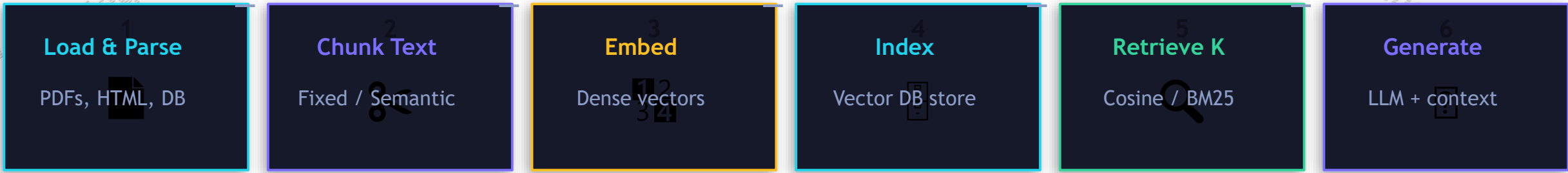
✓ Domain adaptable

RAG Flow



- **User Query:** A user asks a question,
e.g., "What is our policy on international travel?"
- **Semantic Search:** The system searches the external knowledge base (e.g., HR policy documents) for text chunks semantically related to the query.
- **Context Retrieval:** It retrieves the most relevant paragraphs, such as the section on "Overseas Business Trips and Expense Approval."
- **Prompt Enhancement:** It creates a new, detailed prompt for the LLM:
 - *"Using the following context: [insert retrieved text here], answer this question: What is our policy on international travel?"*
- **LLM Generation:** The LLM generates a clear, concise answer based *only* on the provided context, preventing hallucination and ensuring accuracy.

Naïve RAG – The Classic Pipeline



Chunking Strategies

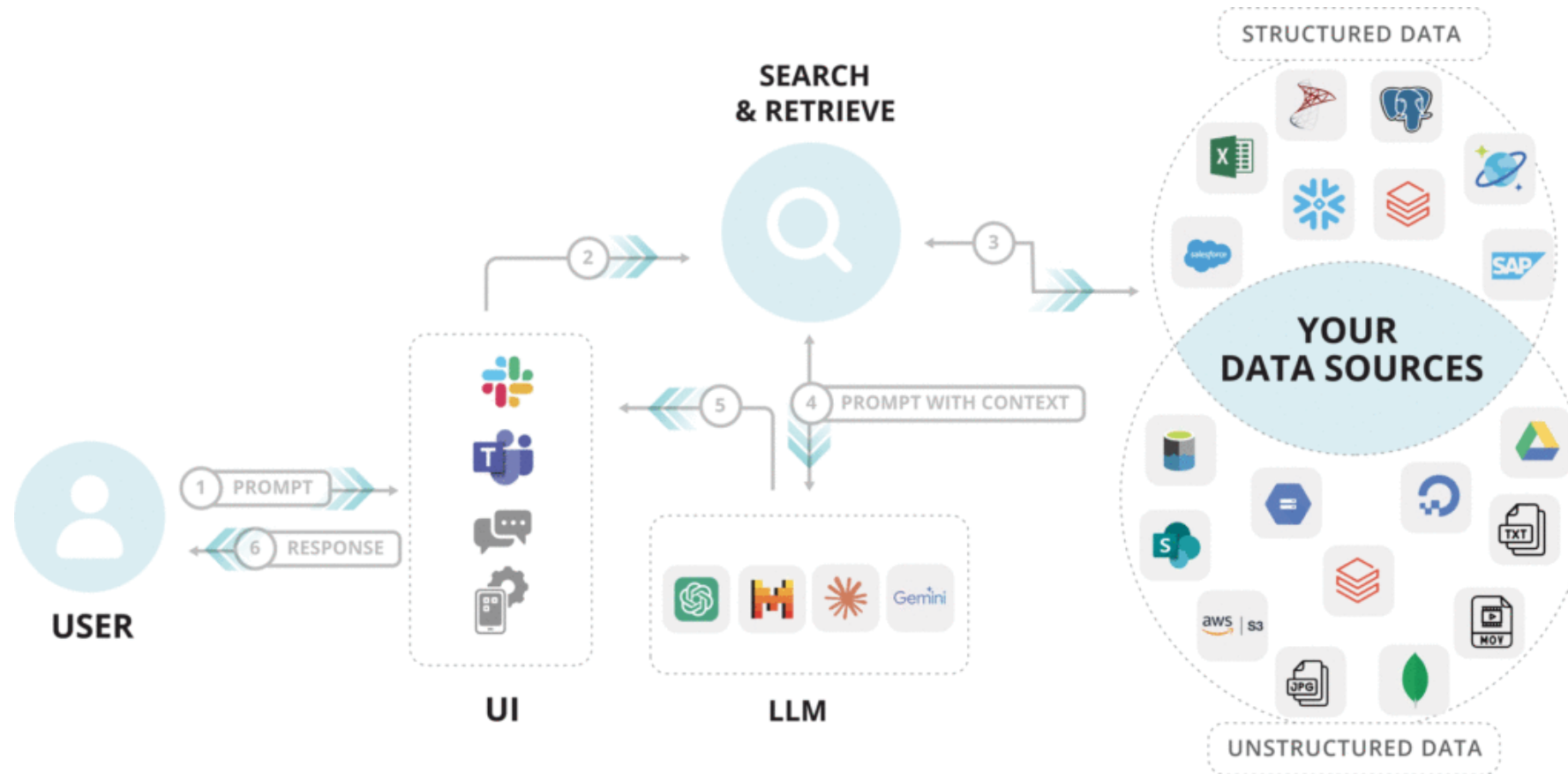
Fixed-size	N tokens with overlap	Simple
Recursive	Para → sentence → word	Semantic
Semantic	Cluster similar embeddings	Coherent
Proposition	Atomic factual decomposition	Precise

Retrieval Techniques

Dense (Bi-encoder)	FAISS cosine / dot-product search
Sparse (BM25/TF-IDF)	Exact keyword term matching
Hybrid + RRF	Fuse dense & sparse with reranking
MMR	Maximal Marginal Relevance diversity

- ⚠ Single-hop only
- ⚠ Flat chunk retrieval
- ⚠ No query planning
- ⚠ Context window limits
- ⚠ No entity linking

How RAG Works



<https://hatchworks.com/blog/gen-ai/rag-for-financial-services/>

Vector RAG



Ingestion: External data (documents, web pages, etc.) is processed, broken into chunks, and converted into numerical representations called **embeddings**.

Indexing: These embeddings are stored in a specialized database called a **Vector Database**.

Retrieval: When a user asks a query, it is also converted into an embedding. The vector database then searches for the most similar data chunks (based on the query embedding).

Augmentation: The original user query and the retrieved data chunks are combined into a comprehensive prompt for the LLM.

Generation: The LLM uses this augmented prompt to generate a response that is informed by the retrieved context.

Vector embedding Search

innovate

achieve

lead

Text Embedding Vectors for Semantic Search

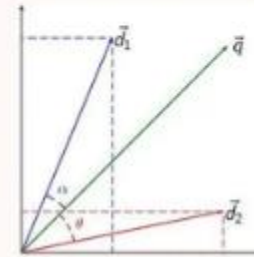
graphsun

Given a question, find the most relevant documents based on a similarity metric (such as Cosine Similarity) between vector of the question and vectors of contents.

Moving from keyword search to similarity (semantic) search.

Q: what is a text embedding?

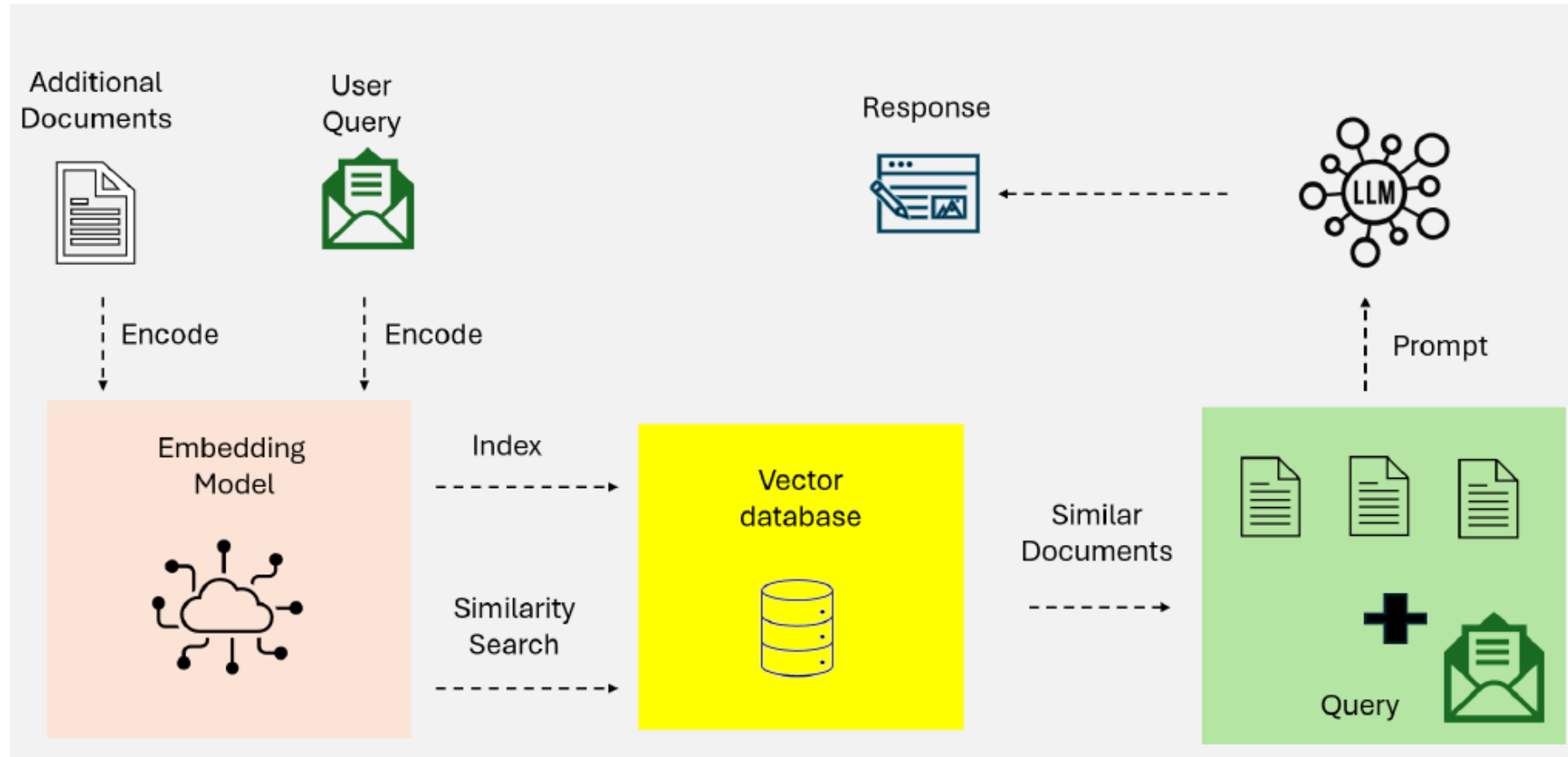
```
embedding [-0.019200351,0.0035306285,0.0  
14364655,-0.009949144,0.00115  
81815,0.017021084,-0.01998375,  
-0.018986698,-0.033643346,-0.0  
2841595,0.014685135,0.0141937  
32,... Show all]
```



Top K by similarity

abstractId	similarity
456	0.923445
22	0.892114
...	...

Vector RAG



<https://arxiv.org/abs/2501.09136>

CODE DEMO RAG

Advanced RAG



While Vector RAG is powerful, it can fall short with multifaceted or complex questions. This has led to the development of more sophisticated RAG architectures:

Graph RAG: Leverages the relationships within structured data (knowledge graphs) for more precise and context-aware retrieval.

Agentic RAG: Introduces autonomous "agents" that can reason, plan, and use tools to improve the retrieval process.

Graph RAG utilizes knowledge graphs—databases that store information as nodes (entities) and edges (relationships)—to enhance retrieval. This is ideal for domains with highly interconnected data, such as finance.

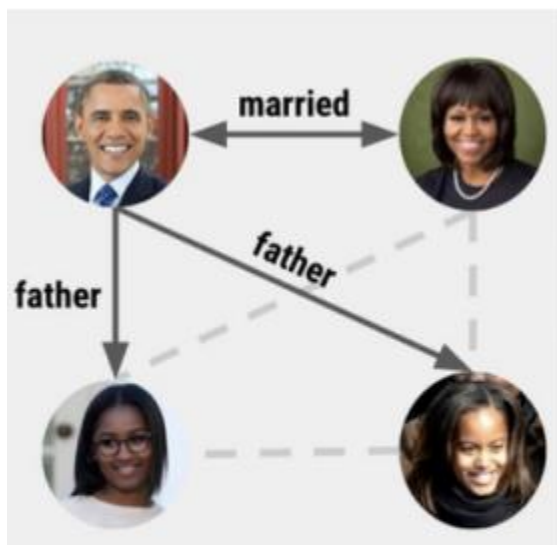
How it Works:

Entity and Relationship Extraction: Key entities and their relationships are identified in the user query.

Graph Traversal: The system navigates the knowledge graph to find relevant nodes and their connections.

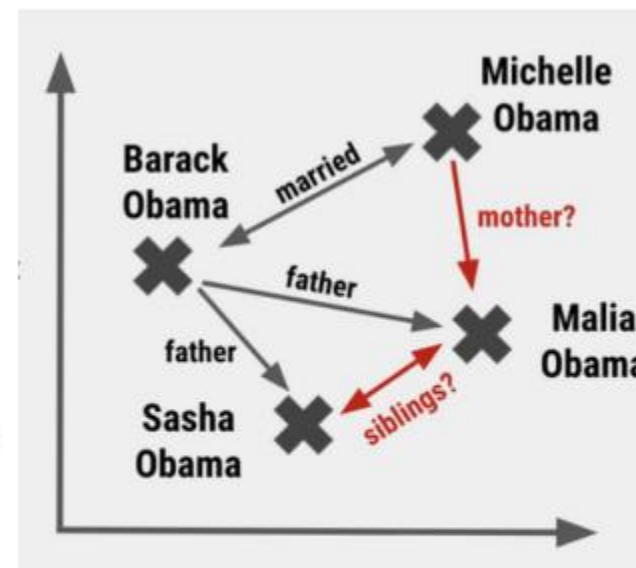
Contextualized Retrieval: Retrieves not just individual facts, but also the surrounding context and relationships.

Knowledge Graph Embedding

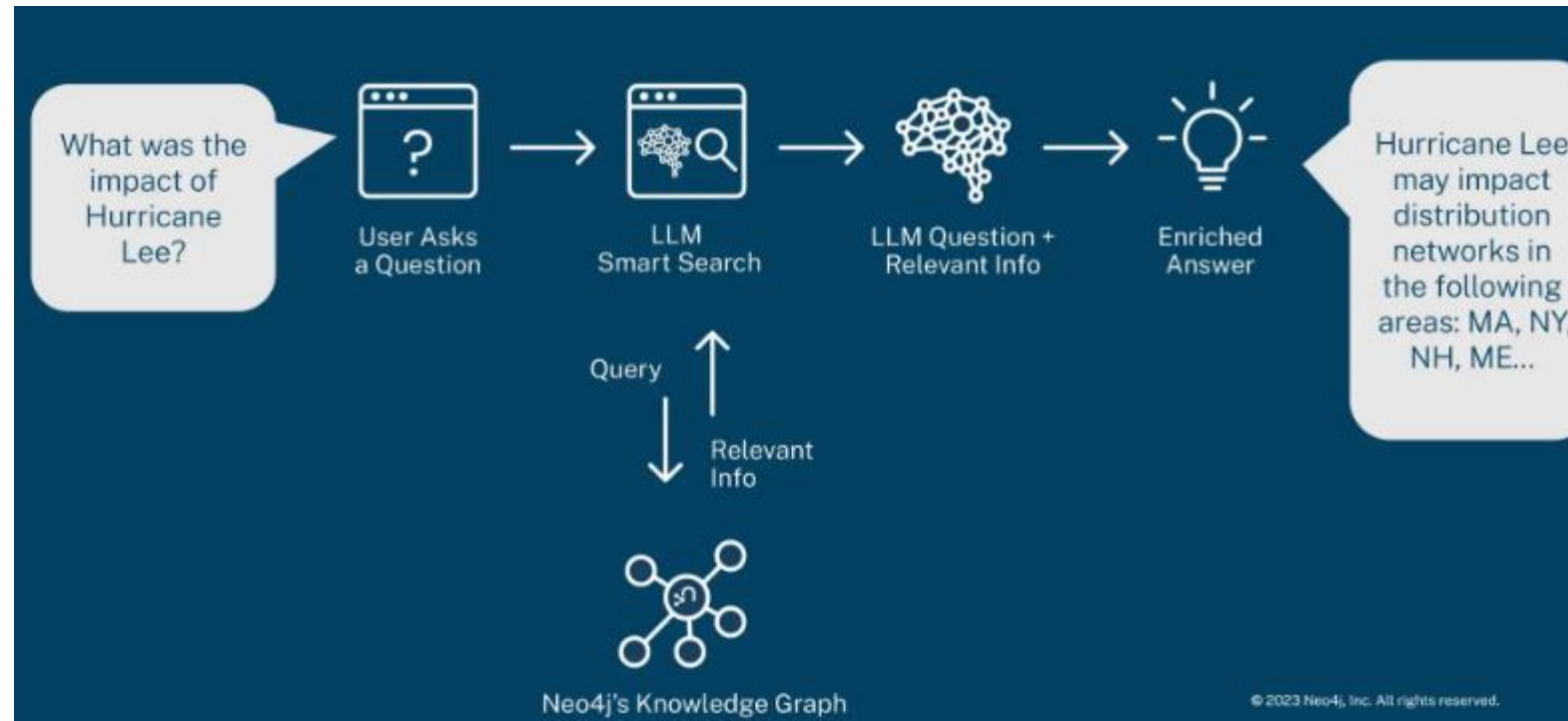


Knowledge graph embeddings:

- TransE, H...
- Neural tensor networks
- Graph convolutional networks (or their variants)
- Matrix factorization
- ...



Knowledge Graph use to ground LLM

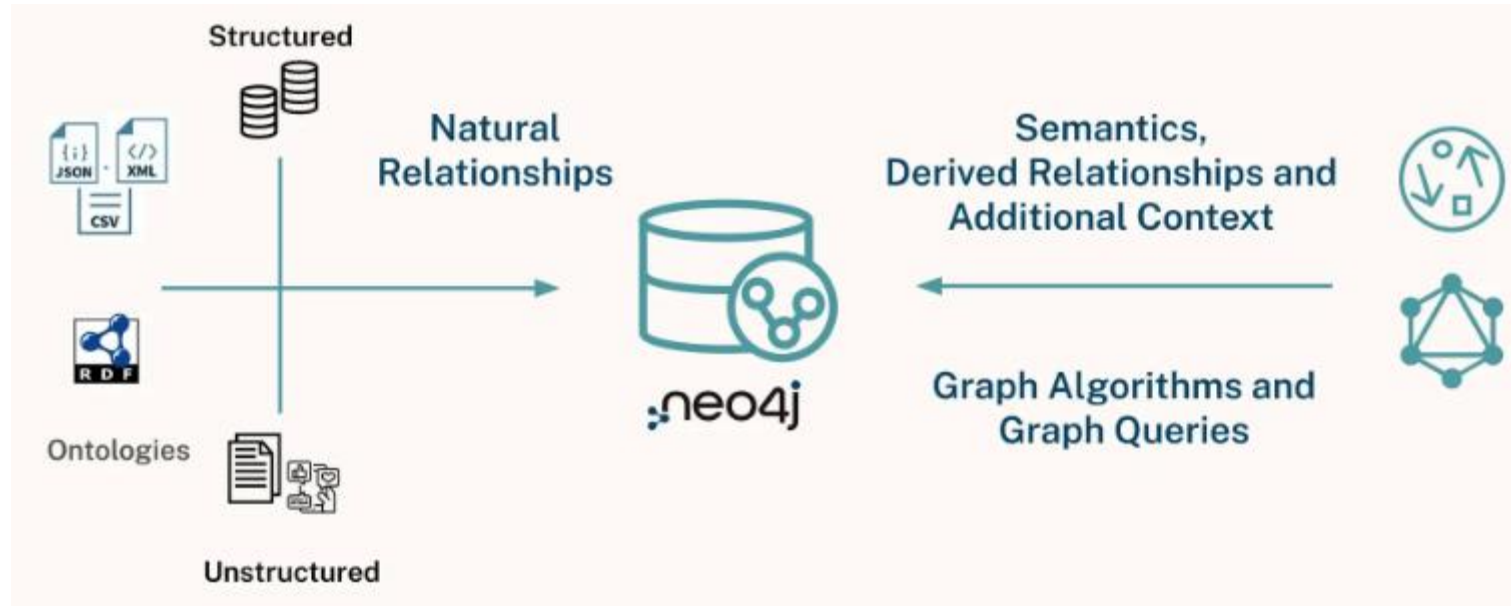


Knowledge Graph

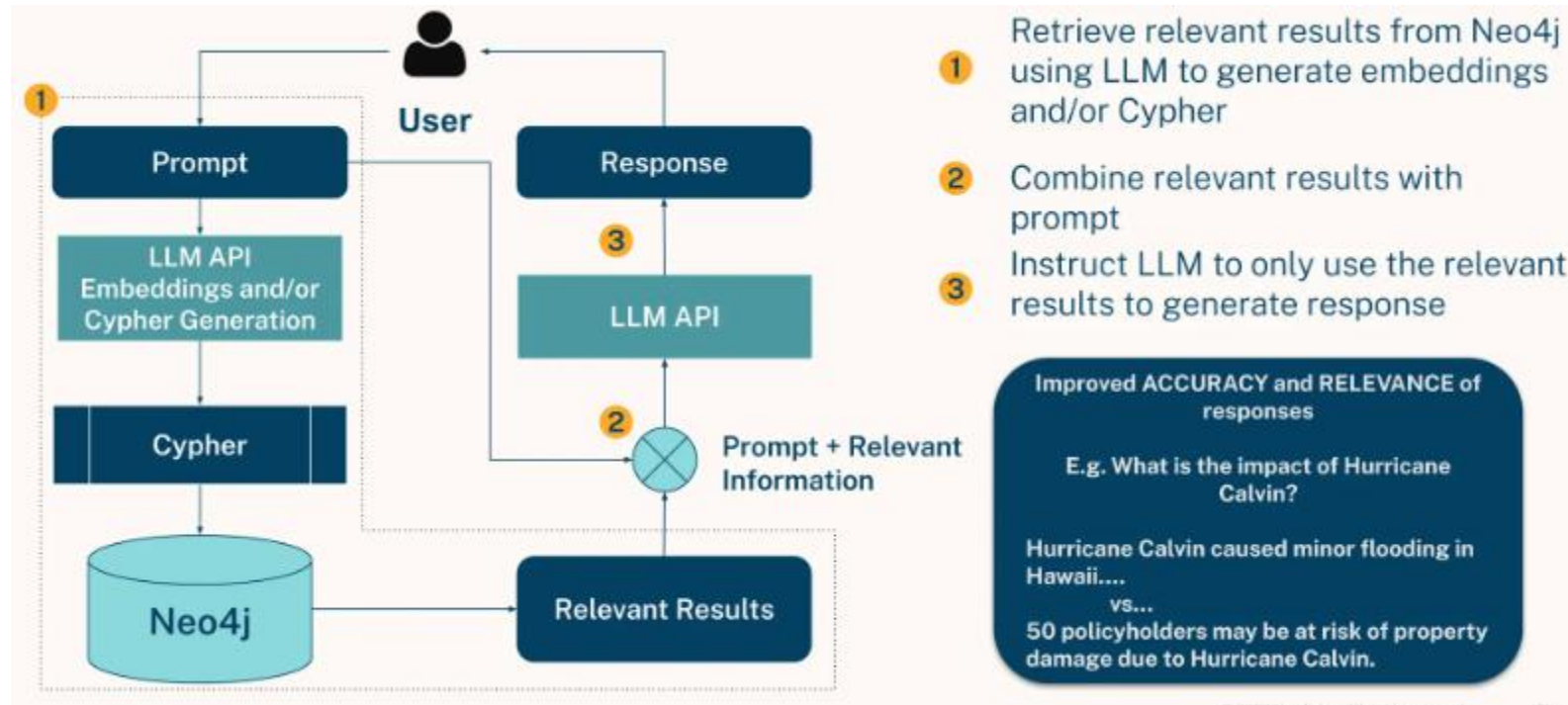
innovate

achieve

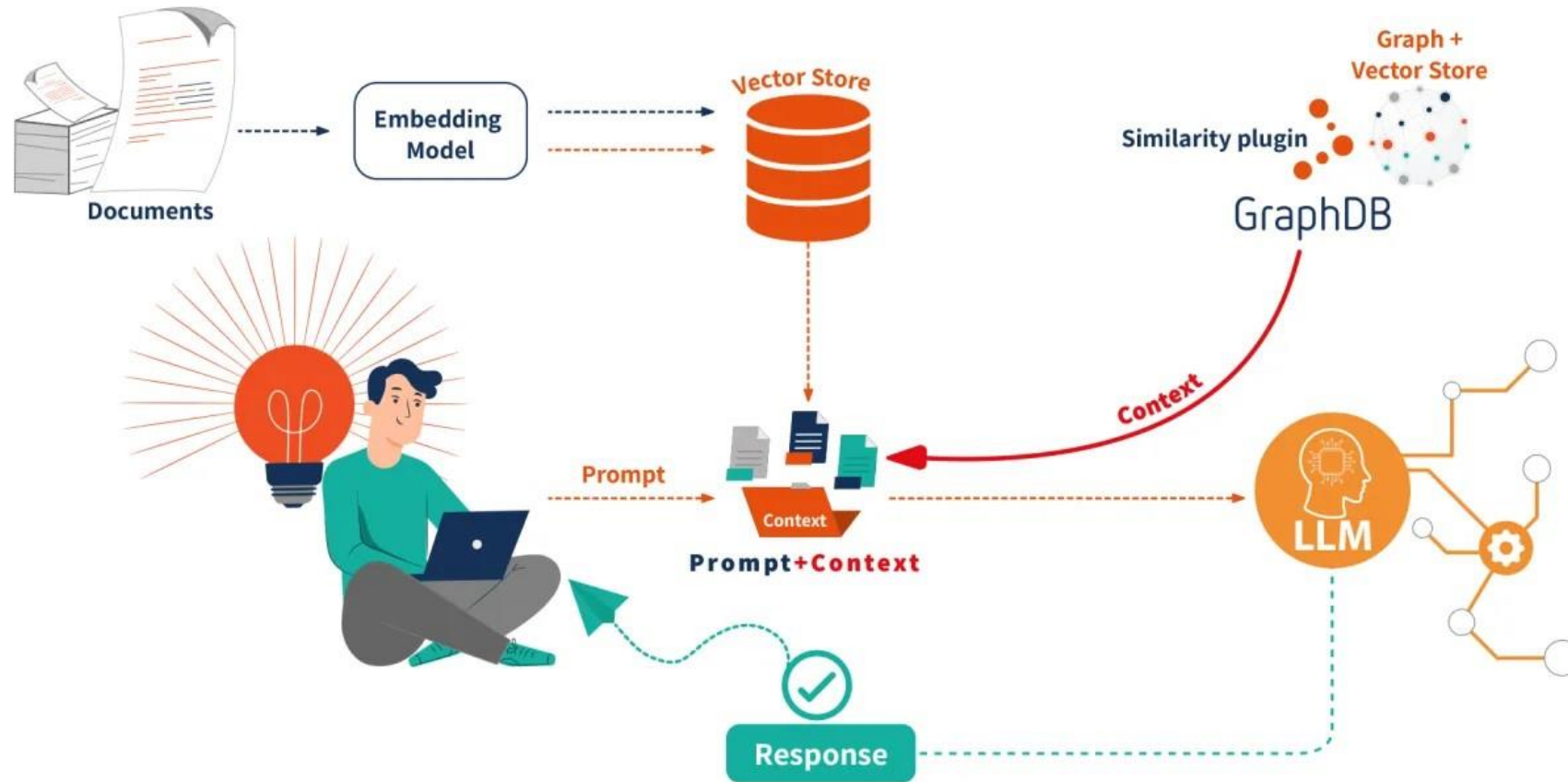
lead



Improve results with knowledge graph using Neo4j



Graph RAG

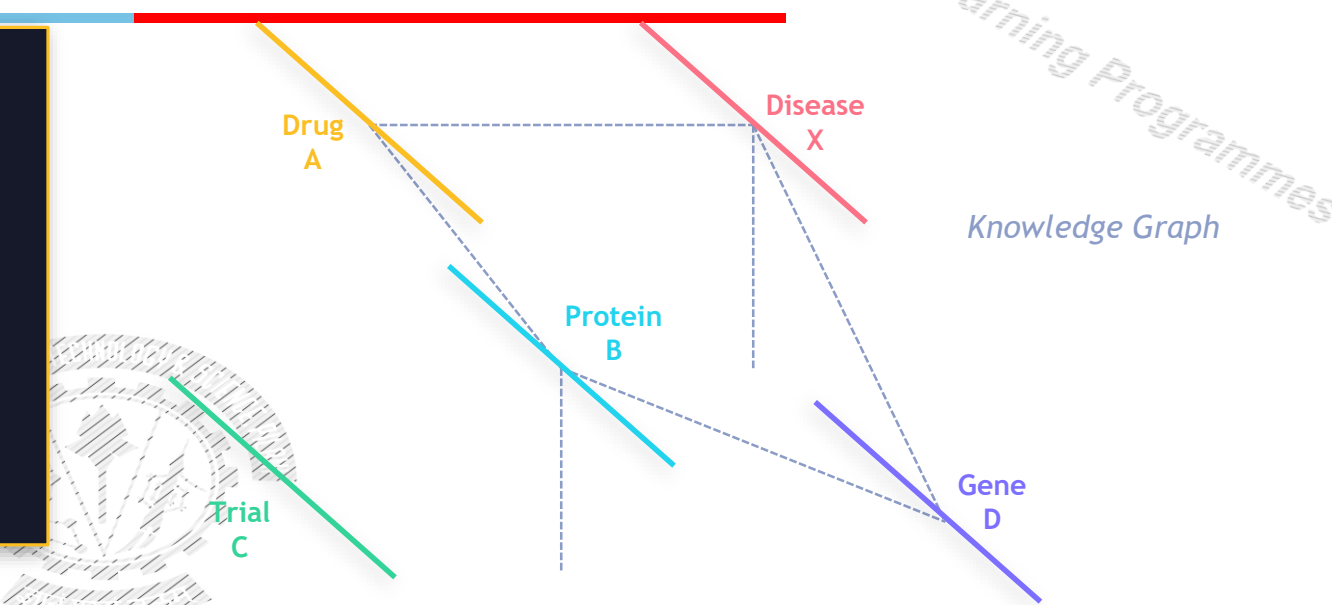


<https://www.ontotext.com/blog/matching-skills-and-candidates-with-graph-rag/>

Graph RAG — Knowledge Graphs Meet LLMs

Why Knowledge Graphs?

- Explicit entity & relationship modelling
- Multi-hop reasoning across graph paths
- Community summaries for global queries
- Structured traversal, not flat similarity
- Handles contradictions via provenance



Graph RAG Architecture (3 Phases)

Phase 1 — Graph Construction

- LLM extracts entities & relations
- Build S-P-O triple store
- Leiden community detection

Phase 2 — Indexing

- Neo4j / Memgraph / NetworkX
- Vector embeddings per node
- Entity resolution mappings

Phase 3 — Query Time

- Classify: local vs global query
- Local → graph traversal
- Global → summary aggregation

📌 *Microsoft GraphRAG (2024) — 3× better global sensemaking vs standard RAG on multi-document corpora benchmarks*

Naïve/Vector RAG vs. Graph RAG

Feature	Naïve RAG	Graph RAG
Data Format	Unstructured Text	Structured Graph
Retrieval Method	Dense Vector Search	Graph Traversal/Querying
Reasoning Capability	Limited	Enhanced (Multi-hop)
Explainability	Low	High
Integration Complexity	Moderate	High
Use Cases	General QA	Domain-Specific Applications

Agentic RAG



Agentic RAG utilizes AI agents to make intelligent decisions about how to best retrieve information. Instead of a linear pipeline, agents can:

- **Decompose Complex Queries:** Break down a broad question into smaller, manageable sub-queries.
- **Iterative Retrieval:** Refine search queries based on initial results to find more relevant information.
- **Tool Use:** Interact with various tools, such as APIs, vector database, graph, or code interpreters, to gather information from diverse sources.
- **Self-Correction:** Analyze the retrieved information for relevance and quality, and re-attempt retrieval if necessary.

This approach is particularly effective for complex research, planning, and problem-solving tasks.

Agentic RAG



A framework where the **LLM acts as an agent**, not just a passive generator.

The agent:

- Sets subgoals.
- Plans retrieval steps.
- Executes tool use (e.g., search, APIs).
- Iteratively refines answers.

Core Idea: Make RAG interactive, iterative, and autonomous.

Agentic RAG



Key Components:

- 1.LLM Agent** – orchestrates reasoning and control.
- 2.Memory / Scratchpad** – stores context, subgoals, past actions.
- 3.Retrievers / Tools** – for search, API calls, calculations.
- 4.Planner / Executor** – decomposes queries, sequences actions.

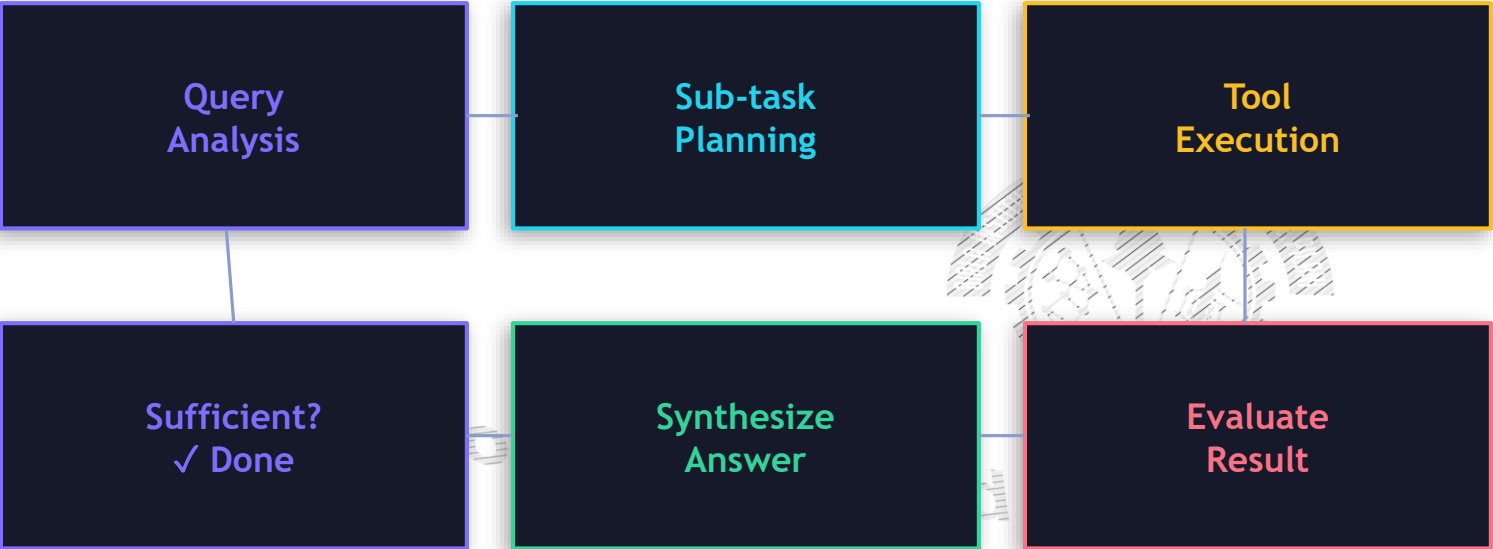
Workflow Example:

User query → Plan → Retrieve → Reason → Iterate
→ Final answer



Agentic RAG – Autonomous Retrieval & Reasoning

The Shift: Static one-shot retrieval → Dynamic, multi-step agents that decide WHAT to retrieve, WHEN, and HOW to synthesize across tools



Key Agentic Patterns

- Corrective RAG**
Self-eval → web search fallback
- Self-RAG**
Reflection tokens: RETRIEVE / ISREL
- Adaptive RAG**
Query routing by complexity class
- Multi-Agent RAG**
Orchestrated specialist agents

Tool Use	Planning	Self-Correct	Memory	Collaboration
APIs, code runners, search engines	Breaks query into decomposed sub-tasks	Re-retrieves when result is insufficient	Short-term context + episodic store	Specialist agents with defined roles

Agentic RAG Example



Query: “Compare the annual energy output of nuclear vs solar in 2024 globally.”

Agentic RAG Flow:

Decomposes into sub-queries.

Searches for energy reports.

Extracts numbers, units, timeframes.

Performs calculations.

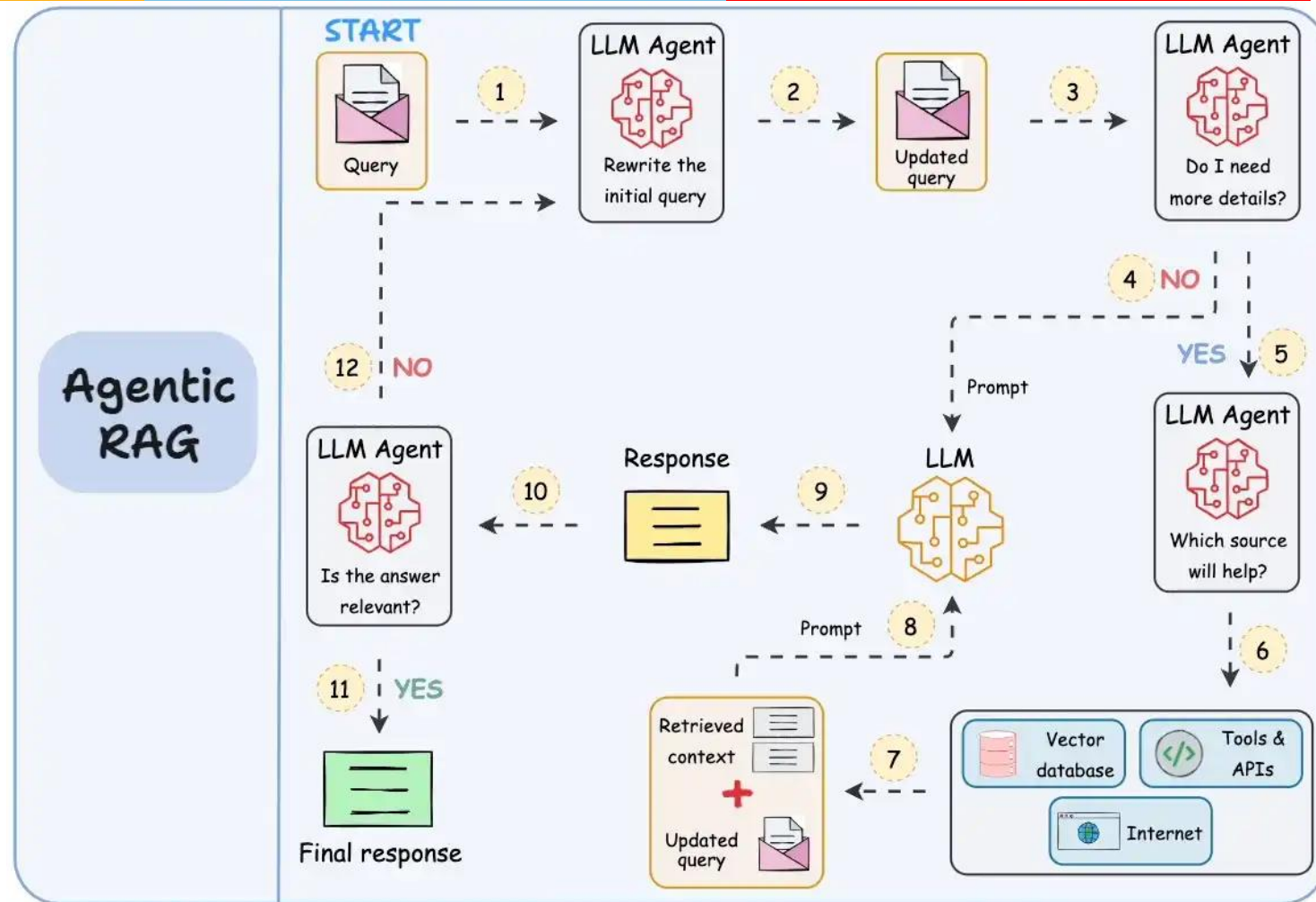
Returns synthesized answer with citations.

Agentic RAG

innovate

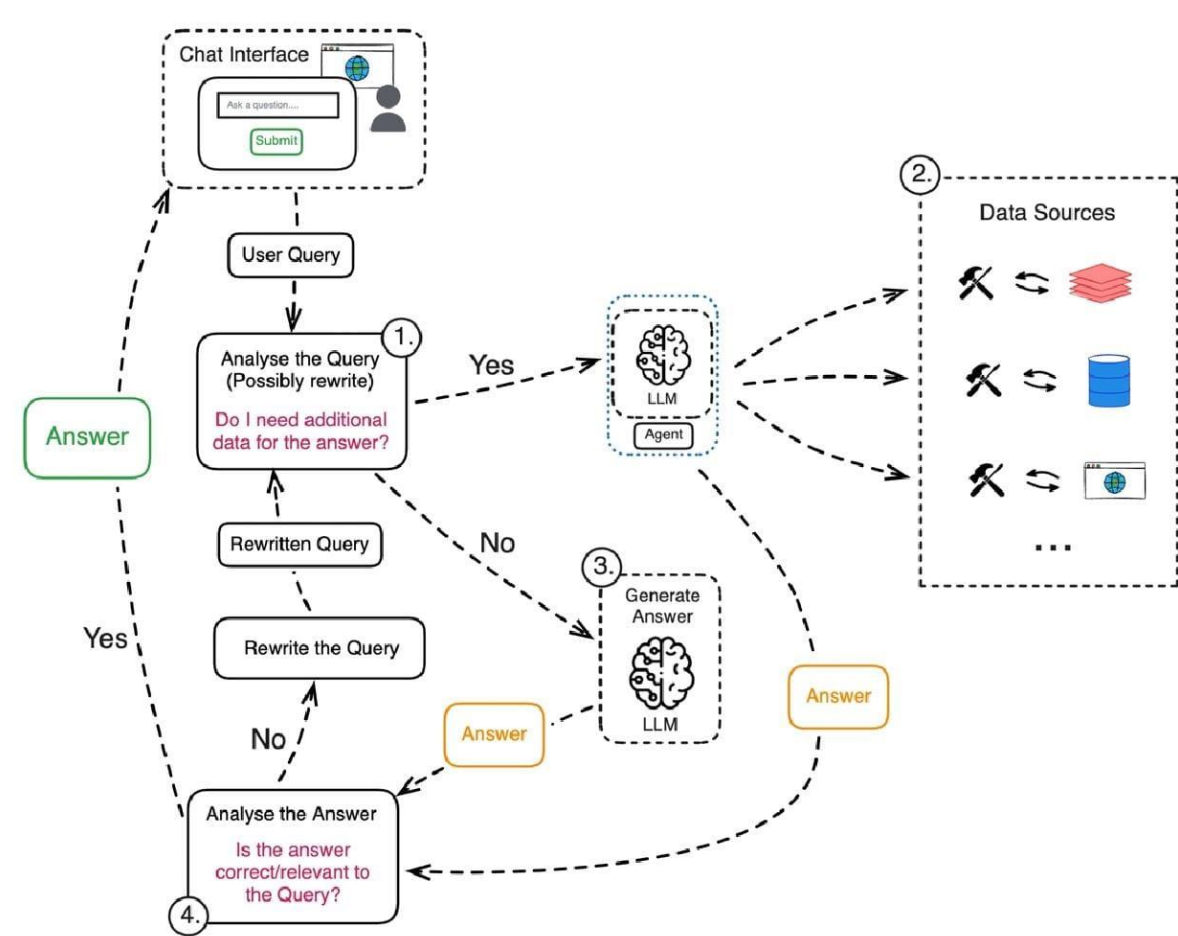
achieve

lead



<https://blog.dailydoseofds.com/p/rag-vs-agentic-rag>

AI Agentic RAG - Example



RAG Variants — Side-by-Side Comparison



Dimension	Naïve RAG	Graph RAG	Agentic RAG
Retrieval Method	Flat vector similarity	Graph traversal + community	Dynamic multi-step tool use
Reasoning Depth	Single-hop only	Multi-hop via entity links	Unlimited iterative loops
Setup Complexity	☆☆ Low	★★★★ High	☆☆☆ Medium-High
Latency	Very low (< 1s)	Medium (graph queries)	Variable (multi-step)
Best NLP Tasks	QA, summarisation, FAQs	Biomedical, legal research	Complex reasoning, research
Traceability	Chunk-level citations	Entity + relation paths	Full agent + tool trace logs
Scalability	Linear with vector DB size	Graph size is bottleneck	Scales with orchestration

Our knowledge isn't just in .txt files. It's in:

- **Images:** Product photos, diagrams in manuals, satellite imagery.
- **Documents:** PDFs containing charts, tables, and complex layouts.
- **Videos:** How-to tutorials, product demos, security footage.
- **Audio:** Customer support calls, meeting recordings.

The Limitation of Standard RAG:

A text-only RAG system is blind to this rich, multimodal information. It can't answer questions like:

- *"Which of our products looks most like this picture?"*
- *"What were the Q3 revenue trends based on this chart in the annual report?"*
- *"Find the part in the video where the presenter explains the system architecture."*

Multimodal RAG



Multimodal RAG is an advanced AI framework that retrieves information from a knowledge base containing multiple data types (modalities) to generate a comprehensive answer.

It understands: Text, images, tables, and potentially audio/video.

It connects: The description of a product with its picture. The text of a report with the data in its charts.

It enables: Users to ask questions using any modality (text or image) and get answers synthesized from all relevant data.

Multimodal RAG



Data Ingestion & Chunking: Documents (like PDFs) are broken down. Text is extracted, images are isolated, and tables are identified.

Multimodal Embeddings: A specialized model creates numerical representations (vectors) for all data types. Critically, it places the vector for an image of a dog near the vector for the word "dog" in a shared "meaning space."

Vector Database: These multimodal embeddings are stored and indexed for fast similarity search.

Multimodal RAG



Hybrid Retrieval: When a user asks a query (text or image), the system converts the query into an embedding and retrieves the most similar text chunks, images, or tables from the vector database.

Generation: The retrieved multimodal context (e.g., a paragraph of text and a relevant chart image) is passed to a powerful Multimodal LLM (like GPT-4o or Gemini) which generates the final, synthesized answer.

Multimodal RAG



Processes More Than Text: It integrates diverse data types like images, audio, and video, not just text, for a richer understanding.

Unified Data Representation: It uses embedding's to create a common language for different data formats, enabling cross-modal comparisons.

Improves Accuracy: By drawing from multiple data types, it gains better context, leading to more accurate and comprehensive answers.

Enables "Any-to-Any" Search: You can use one data type to find another, such as using a text query to retrieve relevant images or vice-versa.

Drives Advanced Applications: This technology powers innovative tools like visual search, interactive education, and complex document analysis.



Multimodal RAG Tools and Technology

Core Technology: Multimodal embedding models (like CLIP) create a unified search space for text, images, and audio.

Infrastructure: Vector databases (e.g., Milvus, Pinecone) are required to store and efficiently query these diverse embeddings.

Orchestration: Frameworks like LangChain and LlamaIndex are used to build and manage the entire RAG pipeline.

Specialized Tools: New open-source projects like Morphik (visual-first) and RAG-Anything (all-in-one) are emerging to solve specific multimodal challenges.

Generation Power: Advanced LLMs like GPT-4o and Gemini simplify the process by directly accepting multimodal inputs for the final response generation.

Enterprise Solutions: Cloud platforms from Google (Vertex AI), AWS, and Azure offer managed services for building scalable, production-grade applications.

Classical vs Multimodal RAG



Multimodal RAG extends classical RAG by indexing and retrieving documents containing **text, images, tables, and video frames** — then fusing those retrieved contexts before generation.

Classical RAG



- ✓ Text chunks only
- ✓ Single-modality index
- ✓ BM25 or dense vectors
- ✓ Text-to-text generation

Multimodal RAG



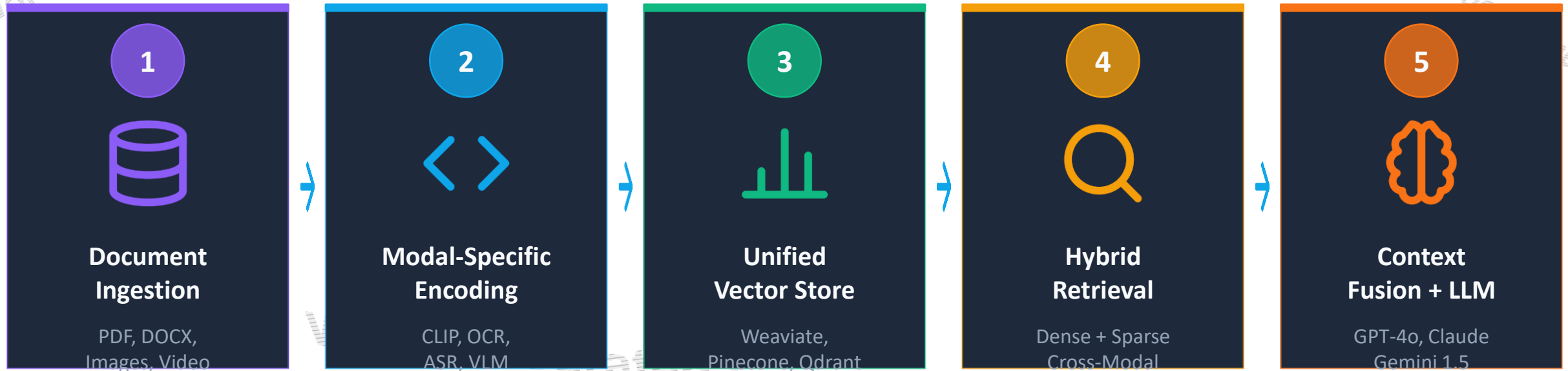
- ✓ Text + Images + Video
- ✓ Joint embedding space
- ✓ Cross-modal retrieval
- ✓ Grounded multimodal output

Key Enablers



- ✓ CLIP / SigLIP encoders
- ✓ Vision-language models
- ✓ Multi-vector stores
- ✓ Instruction-tuned LLMs

Architecture Pipeline



Key Insight: Each modality is encoded independently into a **shared semantic space**, enabling cross-modal similarity search at retrieval time.

INDEXING PHASE

QUERY PHASE



Case Study — Healthcare: Clinical Decision Support

Challenge

- 300+ new clinical guidelines per year
- EMR data siloed from latest research
- Drug interaction DB outdated by months
- High cognitive load → diagnostic errors

RAG Solution

- Graph RAG over PubMed + clinical guidelines
- Real-time retrieval from patient EMR context
- Drug-interaction knowledge graph traversal
- HIPAA-compliant on-premise vector store

Measurable Outcomes

34% reduction in adverse drug events

60% faster clinical guideline lookup

2.1× improvement in evidence citation rate

92% clinician satisfaction (NPS survey)

Tech Stack: LangChain · Neo4j Graph RAG · GPT-4 · FHIR API Connector



Case Study: RAG in the Financial Sector

Problem: Financial analysts need to process vast amounts of unstructured data (e.g., earnings call transcripts, SEC filings, news articles) to make informed investment decisions. This process is time-consuming and prone to human error.

Solution: An investment firm implemented an **Agentic RAG** system to create an "AI Research Assistant."

Key Goals:

Accelerate the analysis of financial documents.

Uncover hidden insights and relationships within the data.

Provide analysts with concise, data-driven summaries and answers to complex financial questions.

Case Studies — Legal & Financial Services



LEGAL — Contract Intelligence Platform

Setup

- Agentic RAG over 50k+ KB articles
- Hybrid BM25 + dense retrieval on clauses
- Jurisdiction-aware metadata filtering
- Citation trail for every finding

Results

80%	manual review time reduction
99.2%	clause extraction accuracy
\$2.4M	annual cost savings per firm
6hr→20min	per 500-page contract

Stack: LlamaIndex · Cohere Reranker · pgvector · Claude 3

FINANCE — Investment Research Automation

Setup

- Multi-agent RAG over SEC, Bloomberg, news
- Parallel async retrieval across 4 sources
- Cross-document reconciliation for conflicts
- Confidence scoring on every data point

Results

75%	faster research report generation
4×	more sources analysed per query
SEC-auto	compliance citation automation
15%	alpha improvement in backtests

Stack: LangGraph · Pinecone · GPT-4o · Bloomberg API



Case Studies – Customer Service & E-Commerce

🎧 CUSTOMER SERVICE – Intelligent Support Agent

Architecture

- Agentic RAG over 50k+ support KB articles
- Live order + inventory data retrieval
- Confidence-based escalation routing
- 32-language multilingual retrieval

Business Impact

68% automated resolution rate

41% CSAT improvement

12 sec mean handle time (was 1.8min)

\$18M annual support cost saved

🛒 E-COMMERCE – Personalised Product Discovery

Architecture

- Multimodal RAG: text + image embeddings
- Real-time catalog sync – no retraining
- User history injection for personalisation
- Inventory-aware retrieval filters

Business Impact

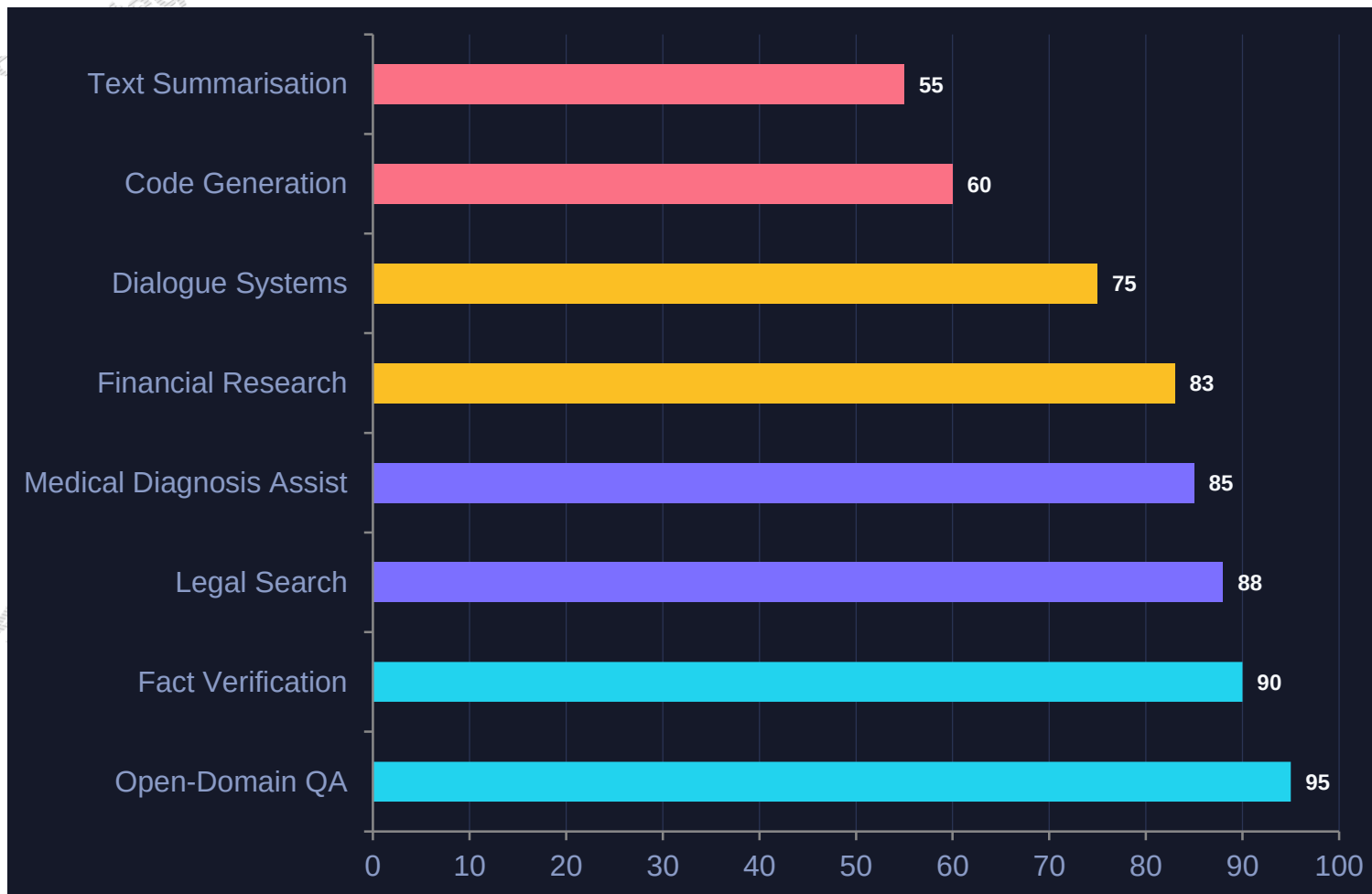
3× click-through rate improvement

22% average order value increase

45% fewer zero-result searches

<200ms p95 retrieval latency

RAG Across NLP Tasks – Where RAG Excels



Relative RAG Benefit Score (Expert Assessment)

NLP Task × RAG Variant Map

Question Answering

Naïve RAG

Multi-document open-domain QA, NQ, TriviaQA

Fact Verification

Graph RAG

FEVER benchmark – entity link verification

Dialogue & Chatbots

Agentic RAG

Session memory + real-time knowledge retrieval

Summarisation

Naïve RAG

Multi-document summarisation with citations

Information Extraction

Graph RAG

Triple extraction from unstructured text

Code Generation

Agentic RAG

Tool-augmented repo retrieval + execution

Case Study - Cybersecurity

innovate

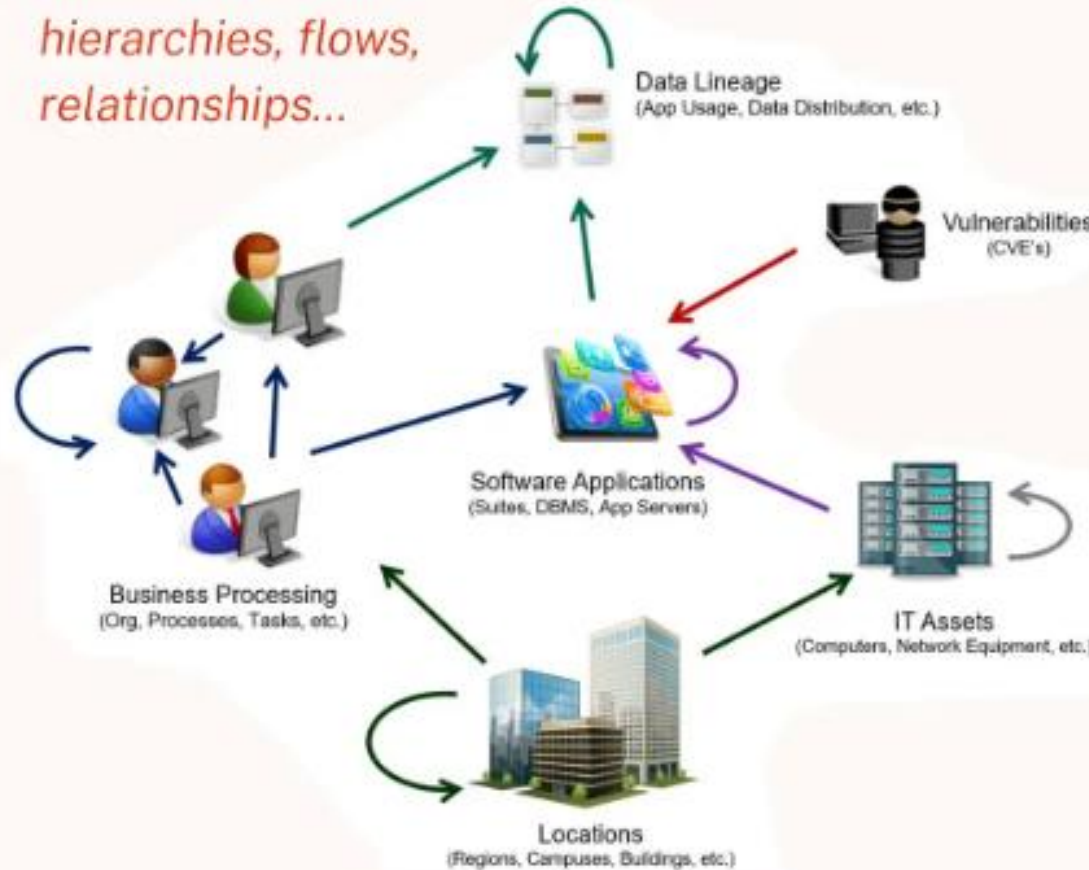
achieve

lead

Analyze business impact of

- software & OS vulnerabilities,
- hardware & software upgrades,
- building/geographic disasters
- changes to business data formats

...across mission critical applications and business locations



Cybersecurity Agent Systems



- Companies like CrowdStrike & Palantir build **threat intelligence graphs**:
 - IPs
 - Malware signatures
 - Actors
 - Campaign timelines
- **Agents**:
 - Detect new threat patterns
 - Trigger automated alerts
 - Suggest mitigation steps



Technology and Tooling Ecosystem

Orchestration Frameworks:

- **LangChain & LlamaIndex:** Provide the core components and abstractions for building RAG pipelines.

Embedding Models:

- **OpenAI, Cohere, Hugging Face:** Provide pre-trained models for converting text into high-quality embeddings.

Large Language Models (LLMs):

- **GPT series (OpenAI), Claude series (Anthropic), Gemini (Google):** The generative powerhouses that produce the final response.

Monitoring and Evaluation:

- **Arize AI, TruEra:** Platforms for monitoring the performance and accuracy of RAG systems in production.



Technology and Tooling Ecosystem

Vector RAG:

Open-Source: LangChain, LlamaIndex, with vector stores like Milvus, Qdrant, Chroma, FAISS and Weaviate.

Commercial: Pinecone, Cohere's RAG, and managed services on clouds like Vertex AI Search.

Graph RAG:

Open-Source: Neo4j with LangChain/LlamaIndex integrations, Diffbot, and custom knowledge graphs.

Commercial: NebulaGraph, Microsoft's GraphRAG, and knowledge graph platforms like Kensho.

Agentic RAG:

Open-Source: LangGraph, CrewAI, and AutoGen, which orchestrate complex RAG workflows.

Commercial: Cognosys, Glean, and various "AI agent" platforms

Understanding the Bottlenecks for Performance Optimization



Embedding Model Speed: The time it takes to convert the user query into an embedding.

Vector Search Latency: The speed of the retrieval from the vector database. This is influenced by the number of vectors, the dimensionality of the embeddings, and the indexing strategy.

LLM Generation Time: The time taken by the LLM to process the prompt and generate a response. This is heavily dependent on the size of the model and the length of the generated output.

Network Latency: The time for data to travel between the different components of the system.

Complexity of Advanced RAG: Agentic and Graph RAG can introduce additional computational steps (e.g., query decomposition, graph traversal), increasing overall latency.

KG role in real world applications

Domain	Application	KG Role
RAG	Graph-RAG, entity-guided retrieval	Structuring retrieval & reasoning
Search/QA	Semantic search, factual QA	Structuring answer space
Conversational AI	Context tracking, disambiguation	Tracking entities and relations
Recommendation	Product/movie suggestions	Semantic user-item modeling
Biomedical	Drug discovery, literature synthesis	Relationship modeling
Fraud/Finance	Anomaly detection	Graph-based behavior patterns
Manufacturing	Digital twins, predictive maintenance	Asset and sensor modeling
XAI	Explainable AI	Traceable reasoning via graph paths
Agentic AI	Planning and decision-making	Knowledge state and goal representation

Real World Applications



- Biomedical Research:** Integrating UMLS KGs with LLMs for accurate medical Q&A.
- Legal Analysis:** Combining legal KGs with LLMs to assist in case law research through structured data retrieval.
- E-commerce:** Enhancing product recommendations and customer support by merging product KGs with LLMs.
- Education:** Personalizing learning experiences using educational KGs and LLMs.
- Financial Reporting:** Analyzing annual financial statements using KG-RAG frameworks.

RAG Strategies Applications



- **Basic RAG**

- if your data is small and questions are simple

- **Hybrid + reranker RAG**

- if accuracy matters

- **Multi-step agentic RAG**

- if questions need reasoning over several documents

- **Structured RAG**

- if data → tables or APIs

- **Graph RAG**

- if relationships are important

RAG Impact



Increased Efficiency: Reduced the time spent on preliminary research

Deeper Insights: The ability to query across both unstructured text and a structured knowledge graph enable analysts to identify trends and risks that were previously missed.

Improved Accuracy: Grounding the LLM's responses in specific financial documents significantly reduces the risk of factual errors in reports.

Competitive Advantage: Able to react more quickly to market-moving information, leading to more timely and profitable investment decisions.

Key Challenges



Data Quality and Cleaning: The performance of any RAG system is highly dependent on the quality of the source data.

Chunking and Embedding Strategy: Finding the optimal way to break down and represent data is crucial for effective retrieval.

Evaluation and Monitoring: Continuously evaluating the relevance of retrieved documents and the accuracy of generated responses is a complex but essential task.

Measuring RAG Quality – RAGAS Evaluation Framework



RAGAS Core Metrics

Faithfulness



Claims in answer supported by context

Answer Relevancy



Answer pertinence to query

Context Recall



Ground truth coverage by chunks

Context Precision



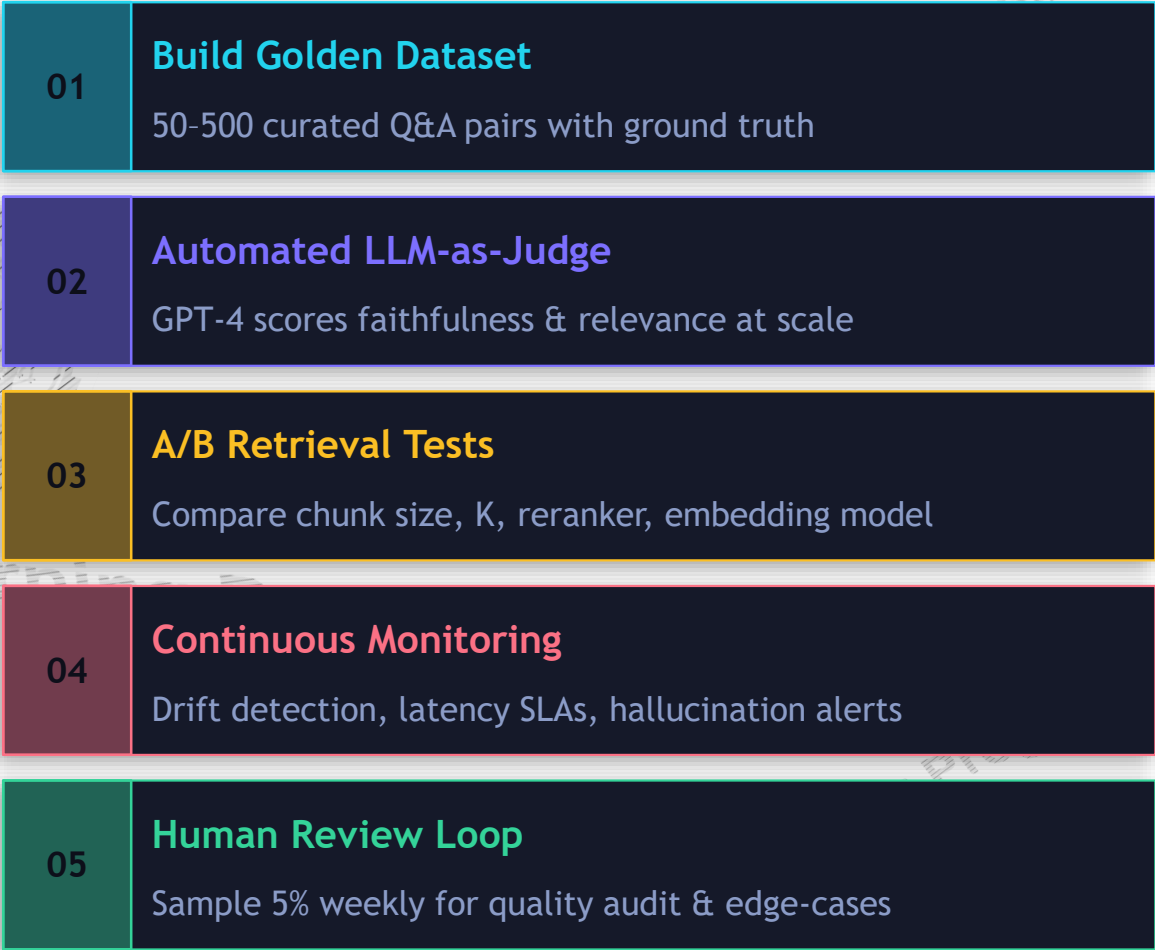
Signal-to-noise in retrieved context

Answer Correctness



End-to-end factual correctness

Evaluation Pipeline



Building Production-Grade RAG Systems



Evaluate First

- Define RAGAS metrics before writing code
- Build 100+ Q&A golden dataset
- Baseline against naive retrieval



Data Quality

- Deduplicate & clean source documents
- Enrich with metadata (date, author, domain)
- Monitor freshness & auto re-index



Performance

- Semantic caching for repeated queries
- Async parallel multi-index retrieval
- Quantised embeddings for 4-10× speed



Security

- Row-level security in vector DB
- PII scrubbing with NER before indexing
- Audit log every retrieval + generation



Observability

- Trace every retrieval with span IDs
- Alert on hallucination score drift
- Dashboard: latency, recall, faithfulness



Continuous Learning

- User feedback → fine-tune retriever
- Re-rank model trained on click-through
- Scheduled golden dataset refresh

Key Takeaways



01

Start with Naïve RAG

It covers 80% of use cases. Optimise chunking, embedding and retrieval before adding complexity.

02

Graph RAG for Relationships

When data has rich entity links or requires multi-hop reasoning, Graph RAG clearly outperforms flat vector search.

03

Agentic for Complex Queries

Planning + tool use + iterative retrieval unlocks reasoning that flat RAG simply cannot match.

04

Evaluate Before You Build

RAGAS metrics + golden dataset first. What you measure is what you improve – always.

05

Production $\neq$ Demo

Caching, security, freshness, latency, and observability separate PoC demos from enterprise grade.

🚀 Build → Measure → Iterate. RAG is an engineering discipline, not just a prompting trick.

Latest RAG Research



2024	GraphRAG (Microsoft) <i>arXiv 2404.16130</i> Hierarchical community detection + LLM summarisation. 3× gain on global sensemaking queries.	2024	CRAG – Corrective RAG <i>ICLR 2024</i> Self-evaluator scores retrieved docs → triggers web search fallback when confidence < threshold.
2024	LightRAG <i>arXiv 2410.05779</i> Dual-level graph retrieval (local + global) with incremental indexing for production workloads.	2024	MiniRAG <i>arXiv 2501.06713</i> Efficient RAG for small LLMs. Heterogeneous graph index + semantic-aware text chunking.
2025	HippoRAG 2 <i>arXiv 2502.12962</i> Neuro-inspired memory with OpenIE triple extraction + PPR-graph scoring for multi-hop QA.	2025	Survey: Agentic RAG <i>arXiv 2501.09136</i> Comprehensive taxonomy of single-agent, multi-agent, and graph-integrated RAG architectures.

Open: RAG evaluation metrics

Open: Chunk boundary optimisation

Open: Privacy in retrieval

Open: Multimodal RAG

Open: Real-time graph updates

References



- Singh, Aditi, et al. "Agentic retrieval-augmented generation: A survey on agentic rag." *arXiv preprint arXiv:2501.09136* (2025).
- **"Retrieval-Augmented Generation (RAG)."** *Google Cloud Technical Documentation*.
- McKinsey & Company "The economic potential of generative AI: The next productivity frontier."
- **RAG for Finance: Automating Document Analysis with LLMs**
- **"Retrieval Augmented Generation (RAG)."** *Pinecone Learning Center*.
- Analysts Explore How Retrieval-Augmented Generation is Emerging as a Key Strategy for Deploying GenAI Applications at the Gartner Data & Analytics Summit
- Need for Contextualized, Reliable and Cost-Effective Solutions Is Driving the Shift Toward Small Task-Specific AI Models
- Financial Analysis: Intelligent Financial Data Analysis System Based on LLM-RAG
- [Future trends in retrieval augmented generation what to expect in 2025 and beyond/](#)
- [RAG Use Cases and Applications](#)
- [The Future of RAG is Agentic - Learn this Strategy NOW](#)
- [Multimodal RAG](#)
- <https://encore.dev/articles/best-vector-databases>

Code References

- https://colab.research.google.com/github/mistralai/cookbook/blob/main/mistral/rag/basic_RAG.ipynb
- <https://colab.research.google.com/github/mrdbourke/simple-local-rag/blob/main/00-simple-local-rag.ipynb#scrollTo=ELIOZ6KTWQga>
- https://colab.research.google.com/github/tomasonjo/blogs/blob/master/llm/enhancing_rag_with_graph.ipynb?ref=blog.premai.io#scrollTo=jPIRSGz4tHNV
- <https://colab.research.google.com/github/microsoft/LLMLingua/blob/main/examples/RAG.ipynb>
- https://colab.research.google.com/github/huggingface/cookbook/blob/main/notebooks/en/agent_rag.ipynb#scrollTo=5Dy-k3Eqxcrl
- <https://colab.research.google.com/github/tomasonjo/blogs/blob/master/bionlp/bioNLP2graph.ipynb>
- <https://encore.dev/articles/best-vector-databases>